URITP Guidebook

1. Introduction

URITP (Universal Robot Instruction/Translation Protocol) is one language and one protocol to program any robot: a wheeled rover, an industrial arm, a drone, a quadruped or a humanoid. You write a program once; URITP checks it against what each robot can actually do, then translates it into that robot's own commands.
This guidebook is the official definition of URITP: its concepts, syntax, types, statements, functions, errors and the rules a URITP-compatible robot must follow. A robot manufacturer should be able to add URITP support to its robots from this document alone.

The vision in one line

Speak to any robot in your own language. An AI layer translates your input into URITP and URITP makes sure the robot understands, and does it safely.

Who uses URITP

- **Everyone** talks to robots in their own language (English, Arabic, Hindi and so on). An AI layer turns speech into URITP.
- **Developers and engineers** write URITP directly when they need precision, and build apps and skills.
- **Manufacturers** add URITP support to their robots by following section 9, and keep their unique features through extensions.

Design principles

1. **Goals, not motions.** You say where a part of the robot should end up; the robot decides how (gait, flight, joint path).
2. **Never silently approximate.** A program either runs with the same meaning on a robot, or is rejected before it starts, with a clear reason.
3. **Check before you move.** Types, units, capabilities and safety limits are verified before any motor turns.
4. **Readable by a beginner.** A school student should read a URITP program and understand it.
5. **Units are part of the language.** `10cm`, `0.2m/s` and `90deg` are values, never bare numbers.
6. **Portable core, open edges.** The core works everywhere; vendor extensions and a native escape hatch cover the rest, clearly marked.
7. **Safety is not optional.** Every program runs inside a safety envelope that code cannot switch off.

Scope

This guidebook defines what every URITP program means and what every URITP-compatible robot must do. It does not tell manufacturers how to build that support inside their robots; each manufacturer does that in its own way, as long as the rules in section 9 are met. URITP sends goals and receives results; the robot's own controller keeps control of its motors.

2. Core concepts

URITP rests on one idea: **a robot is a set of controllable frames in space, each with constraints, and a command is a goal on a frame.** Everything else builds on the ten concepts below.

Concept	Definition	Example
Robot	A connected machine that runs URITP.	`robot arm = connect("sim:panda")`
Frame	A named point on a robot or in the world, with a position and orientation (a pose).	`arm.tool`, `rover.base`, `helper.left_hand`
Constraint	The limits on how a frame can move: which axes, how far, how fast.	a rover's `base` moves on a plane only
Goal	A target for a frame: a pose, a tolerance, optional speed and path rules.	`move arm.tool to cup.above(10cm)`
Actuator	A part with discrete states, attached to a frame.	`grip arm.gripper close`
Sensor	A source of readings about the robot or the world.	`rover.sense("distance_front")`
Task	Every command runs as a task with a lifecycle (below).	a `move` is one task
Safety envelope	Limits that every program runs inside: speeds, zones, stop.	`safety:` block, e-stop
Profile	A standard set of commands shared by a family of robots.	`navigation`, `manipulation`, `flight`
Extension	A vendor-specific feature under the vendor's own namespace.	`examplecorp.adaptive_weld(...)`

Frames across robot types

Robot	Controllable frames	Constraint
Wheeled rover	`base`	plane: x, y, rotation
Industrial arm	`tool`	full 3D pose within reach
Drone	`base`	x, y, z, rotation, in the air
Quadruped	`base`	plane plus terrain it can handle
Humanoid	`base`, `left_hand`, `right_hand`	walking plane plus reach of each arm

Navigation and manipulation are the same operation: navigation moves the `base` frame, manipulation moves a tool frame. A mobile manipulator decides for itself whether to drive,

reach, or both.

Task lifecycle

Every command becomes a task that moves through these states:

1. `accepted`: the robot has taken the goal.
2. `running`: the robot is working on it.
3. One final state: `succeeded`, `failed(reason)` or `cancelled`.

Because failure behaves the same on every robot, `on fail` handlers work identically everywhere.

Capability manifest

Each robot publishes a manifest listing its frames, constraints, actuators, sensors, profiles and extensions. The checker reads it before a program runs. The format is in section 9.

3. Lexical structure

Every URITP file uses the extension `.uritp`: programs, site maps and robot manifests alike. Files are UTF-8 and group code by indentation.

Comments

```
# a line comment
move rover.base to places.dock   # a trailing comment
```

Indentation and blocks

A line ending in `:` opens a block. The block's lines are indented by 4 spaces. Tabs are an error, so indentation always looks the same in every editor. A line that continues a statement (such as `avoid` under `move`) is indented one level deeper than the statement it belongs to.

Identifiers

Letters, digits and `_`, not starting with a digit: `arm`, `pick_and_place`, `box_231`. Dotted names address parts: `arm.tool`, `places.kitchen`, `field.row(4)`. Vendor extensions use the vendor's name as the first part: `examplecorp.adaptive_weld`.

Keywords (reserved)

`robot` `connect` `let` `task` `return` `move` `to` `by` `speed` `tolerance` `via` `avoid` `drive` `turn` `forward` `backward` `grip` `open` `close` `pick` `place` `using` `onto` `on` `pour` `from` `into` `weld` `along` `spray` `amount` `current` `takeoff` `land` `at` `capture` `save` `as` `say` `lang` `confirm` `ask` `alert` `log` `wait` `until` `timeout` `if` `elif` `else` `and` `or` `not` `for` `in` `repeat` `every` `times` `forever` `break` `parallel` `fail` `safety` `when` `stop` `return_home` `native` `import` `true` `false`

Also reserved: `define` (site maps), `force` (grip force), and in manifest files `manifest`, `uritp`,

`frame`, `actuator`, `sensor`, `profiles`, `extension`.

Literals

Kind	Examples	Notes
Number	`3`, `0.5`, `-2`	plain count, no unit
Quantity	`10cm`, `0.2m/s`, `90deg`, `2min`	number immediately followed by a unit, no space
Percentage	`25%`, `80%`	0% to 100%
Text	`"Parcel for you"`	double quotes; `{name}` inside inserts a value
Boolean	`true`, `false`	
List	`[places.a, places.b]`	

Units

Dimension	Units
Length	`mm`, `cm`, `m`, `km`
Angle	`deg`, `rad`
Time	`ms`, `s`, `min`, `h`
Speed	`mm/s`, `cm/s`, `m/s`, `km/h`
Angular speed	`deg/s`, `rad/s`
Mass	`g`, `kg`
Force	`N`
Current	`A`
Volume	`ml`, `l`
Temperature	`deg_c`

Quantities of the same dimension convert automatically (`10cm + 5mm` is `10.5cm`). Mixing dimensions (`10cm + 2s`) is an error, reported before the program runs. Temperature uses `deg_c` so it can never be confused with the angle unit `deg`.

4. Types and values

URITP is statically typed but you rarely write types: the checker infers them. Every value has one of the types below.

Type	What it holds	Example value
`Number`	a plain count or ratio	`3`, `0.5`
`Quantity<D>`	a number with a unit of dimension D (Length, Angle, Time, Speed, Mass, Force, Current, Volume, Temperature)	`10cm`, `0.2m/s`
`Percent`	0% to 100%	`25%`
`Bool`	true or false	`true`
`Text`	a string, optionally with a language tag	`"Hello"`
`Pose`	a position (x, y, z) plus	`pose(1m, 2m, 0m,`

Type	What it holds	Example value
	orientation	`yaw=90deg)`
`Frame`	a controllable or observable frame	`arm.tool`
`Place`	a named pose in the robot's map	`places.kitchen`
`Zone`	a named area, used in `avoid` and safety rules	`zones.icu`
`Route`	an ordered list of places or poses	`route.night_patrol`
`Object`	a thing the robot can perceive and act on	`cup`, `box_231`
`Robot`	a connected robot	the result of `connect(...)`
`Actuator`	a part with discrete states	`arm.gripper`
`Sensor`	a source of readings	`dog.sense("thermal")`
`Reading`	a sensor result, with a timestamp	`.max`, `.value`, `.health`
`List<T>`	an ordered list	`[places.a, places.b]`
`TaskResult`	the final state of a task	`succeeded`, `failed("blocked")`

Rules the checker enforces

- A `Pose`, `Place` or `Object` can be the target of `move`. An `Object` target means "the object's reference frame"; helpers such as `.above(10cm)` give a precise pose.
- Comparisons need matching dimensions: `distance < 20cm` is fine; `distance < 20` is an error (bare number where a Length is expected).
- `Percent` and `Number` never mix silently: `battery < 25%` is correct, `battery < 25` is an error.
- Variables are immutable by default. `let x = 5cm` binds a value; rebinding the same name in the same block is an error, which keeps robot programs predictable.

5. Statement reference

Every URITP statement, grouped by purpose. In the syntax column, `[ ]` marks optional parts and `...` marks a repeatable part. The Profile column says which robots support it (section 7).

Program structure

Statement	Syntax	Profile	Meaning
robot	`robot NAME = connect("ADDRESS")`	core	Connects to a robot and loads its capability manifest.
let	`let NAME = EXPR`	core	Binds a value to a name.
task	`task NAME(PARAMS): +`	core	Defines a reusable routine.

Statement	Syntax	Profile	Meaning
	block		
call	`NAME(ARGS)`	core	Runs a task or built-in function.
return	`return [EXPR]`	core	Ends a task, optionally with a value.
import	`import "FILE.uritp"`	core	Loads tasks, places and zones from another file.

Motion

Statement	Syntax	Profile	Meaning
move	`move FRAME to TARGET [speed Q] [tolerance Q] [via ROUTE]` + optional `avoid ZONE` line	core	Brings a frame to a target pose. The robot plans the motion.
drive	`drive FRAME forward/backward [by Q] [speed Q]`	navigation	Drives straight, for a distance or until stopped.
turn	`turn FRAME by ANGLE [speed Q]`	navigation, legged, flight	Rotates the frame in place.
takeoff	`takeoff ROBOT to HEIGHT`	flight	Lifts off and hovers at the height.
land	`land ROBOT [at PLACE]`	flight	Lands, here or at a place.
stop	`stop ROBOT`	core	Stops all motion immediately (task becomes `cancelled`).

Manipulation

Statement	Syntax	Profile	Meaning
grip	`grip ACTUATOR open/close [force Q]`	manipulation	Opens or closes a gripper or hand.
pick	`pick OBJECT using ROBOT [onto FRAME]`	manipulation	Full grasp sequence: approach, grip, lift, optionally deposit.
place	`place OBJECT on TARGET using FRAME`	manipulation	Puts a held object down at a target.
pour	`pour from OBJECT into OBJECT until CONDITION`	manipulation	Pours while a condition holds.
weld	`weld along PATH`	manipulation + welding	Moves the tool along a

Statement	Syntax	Profile	Meaning
	`speed Q current Q`	tool	seam while welding.
spray	`spray ACTUATOR on TARGET amount Q`	actuator with spray capability	Sprays a set volume.

Sensing and data

Statement	Syntax	Profile	Meaning
sense	`ROBOT.sense("SENSOR")`	core	Returns the latest `Reading` from a sensor.
capture	`capture SENSOR save as "FILE"`	camera sensor	Takes a photo or scan and saves it.
log	`log EXPR, EXPR, ...`	core	Records values to the run log.
alert	`alert "TEXT"`	core	Sends a notification to operators.

Talking to people

Statement	Syntax	Profile	Meaning
say	`say "TEXT" [lang CODE]`	speech	Speaks or displays a message.
confirm	`confirm "QUESTION" [lang CODE]`	speech	Asks yes/no; `no` or no answer cancels the program safely.
ask	`let x = ask "QUESTION" [lang CODE]`	speech	Asks a question and returns the answer as Text.

Control flow

Statement	Syntax	Meaning
if	`if COND:` block, then optional `elif COND:` and `else:`	Runs a block when a condition holds.
for	`for NAME in LIST:` block	Repeats for each item.
repeat	`repeat N times:` / `repeat every Q:` / `repeat forever:`	Loops a fixed number of times, on a schedule, or until stopped.
break	`break`	Leaves the innermost loop.
wait	`wait Q` / `wait until COND [timeout Q]`	Pauses for a time or until a condition. Timeout makes the task fail.
parallel	`parallel:` block	Starts every statement in the block at once and waits for all to finish.

Errors and safety

Statement	Syntax	Meaning
on fail	`on fail:` block, at the end of a block or task	Runs if any task in the enclosing block fails. The variable `error` holds the reason.
safety	`safety:` block of rules	Declares rules checked continuously while the program runs.
when	`ACTION when COND` (inside `safety:`)	Triggers `stop`, `return_home`, `land` or `alert` when a condition becomes true.
native	`native("LANGUAGE", "CODE")`	Escape hatch: sends raw robot-specific code. Marked non-portable; the checker warns.

Control-flow and error statements belong to `core` and work on every robot.

6. Standard library

Built-in functions and namespaces available in every program.

Pose helpers (on any Pose, Place or Object)

Function	Returns	Meaning
`.above(Q)`	Pose	Same position, raised by a length, tool pointing down.
`.below(Q)`	Pose	Lowered by a length.
`.beside(Q)`	Pose	Offset sideways by a length, facing the target.
`.in_front(Q)`	Pose	Offset along the target's forward direction.
`.at_height(Q)`	Pose	Same x, y at an absolute height.
`.offset(dx, dy, dz)`	Pose	Offset by three lengths in the world frame.
`.handle`	Pose	The grasp point of an object, if its model defines one.
`.level(N)`	Pose	The Nth level of a shelf-like object.
`pose(x, y, z, roll=, pitch=, yaw=)`	Pose	Builds a pose from lengths and angles.
`distance(a, b)`	Length	Straight-line distance between two poses.

World namespaces

Namespace	Holds	Defined in
`places.NAME`	named poses	the site map file (`site.uritp`) or the robot's own map
`zones.NAME`	named areas	the site map file
`route.NAME`	ordered lists of places	the site map file
`field.row(N)`	rows of an agricultural field, with `.plants`	a field map
`shelf.ID`	storage locations	warehouse map

A site map file declares them in URITP itself:

```
define place kitchen = pose(4.2m, 1.0m, 0m, yaw=90deg)
define place door    = pose(0m, 3.5m, 0m)
define zone icu      = polygon(pose(10m,0m,0m), pose(14m,0m,0m),
pose(14m,6m,0m), pose(10m,6m,0m))
define route night_patrol = [places.kitchen, places.door]
```

(`define` is used here so it never clashes with the `place` statement.)

Robot properties (on any Robot)

Property	Type	Meaning
`.battery`	Percent	Current charge.
`.state`	Text	`idle`, `busy`, `error` or `stopped`.
`.pose`	Pose	Where the base frame is now.
`.has("NAME")`	Bool	True if the robot supports a profile or extension.
`.frames`	List	The robot's controllable frames.

Reading properties

`.value`, `.max`, `.min`, `.mean`, `.timestamp`. Vision readings add `.analyse(OBJECT)`, which returns a result with fields such as `.health` or `.label`, depending on the model attached.

Actuator and object properties

`gripper.holding` (Bool), `tray.loaded` (Bool), `drawer.locked` (Bool), `object.fill` (Percent), `object.id` (Text), `object.name` (Text).

Other built-ins

Function	Meaning
`now()`	Current time.
`len(list)`	Number of items.
`round(q, unit)`	Rounds a quantity in a given unit.
`to(q, unit)`	Converts a quantity, e.g. `to(1.2m, cm)` gives `120cm`.
`text(v)`	Converts a value to Text.

7. Profiles and extensions

URITP has three layers of features. A program that uses only the first two runs on any robot with those profiles; the third is always marked.

Layer 1: Core (every robot)

`robot`, `let`, `task`, `move`, `stop`, `wait`, `if`, `for`, `repeat`, `parallel`, `on fail`, `safety`, `sense`, `log`, `alert`, plus the whole standard library. Every URITP-compatible robot must implement all of it.

Layer 2: Standard profiles (by robot family)

Profile	Adds	Typical robots
`navigation`	`drive`, `turn`, `via`, `avoid`, places and routes	wheeled rovers, AMRs, forklifts
`manipulation`	`grip`, `pick`, `place`, `pour`, object poses	arms, humanoids, mobile manipulators
`flight`	`takeoff`, `land`, 3D base goals, `return_home`	drones
`legged`	terrain-aware base goals, `turn`	quadrupeds, humanoids
`speech`	`say`, `confirm`, `ask`, `lang`	service and home robots
`vision`	`capture`, `.analyse(...)`, object detection	any robot with a camera
`tooling`	`weld`, `spray` and other tool actions	industrial and farm robots

A robot declares its profiles in its manifest. A humanoid might declare `[core, navigation, legged, manipulation, speech, vision]`.

Layer 3: Vendor extensions

Unique features live under the vendor's own namespace, so they never clash:

```
if welder.has("examplecorp.adaptive_weld"):
    examplecorp.adaptive_weld(seam, current=180A)
else:
    weld along seam speed 5mm/s current 180A
```

Rules for extensions:

1. Name them `vendor.feature`, lower case.
2. List them in the manifest with their parameters and types, so the checker can verify calls.
3. Programs should guard them with `.has(...)` and provide a fallback. The checker warns if a call has no guard.

Promotion: how the standard grows

When several vendors ship similar extensions, the URITP maintainers merge them into a standard profile. Vendor features that prove useful across the industry become part of URITP,

the same way graphics APIs grew from vendor extensions.

The native escape hatch

`native("urscript", "set_tcp(p[0,0,0.12,0,0,0])")`
Sends raw code in the robot's own language. The checker marks the program non-portable and prints a warning. Use it only when no core feature, profile or extension covers the need.

8. Errors and the checker

URITP catches most mistakes before a robot moves. Errors stop the program from running; warnings let it run but are always shown.

Code	Stage	Catches	Example message
E1xx	Syntax	typos, bad indentation, missing `:`	`E101 (line 4): expected ':' after 'if' condition`
E2xx	Names	unknown robots, frames, places, tasks	`E203 (line 3): robot 'rover' has no frame 'tool'`
E3xx	Types and units	wrong types, mixed dimensions, bare numbers	`E304 (line 7): expected Length, got Number '20'. Did you mean 20cm?`
E4xx	Capabilities	commands the robot cannot perform	`E401 (line 2): 'takeoff' requires profile 'flight'; rover supports [core, navigation]`
E5xx	Safety	limits and zones that would be broken	`E502 (line 5): speed 3m/s exceeds safety limit 1.5m/s for 'arm'`
W6xx	Portability	extensions without guards, `native`	`W601 (line 9): 'examplecorp.force_guide' used without welder.has(...) guard`
R7xx	Runtime	failures while running, delivered to `on fail`	`R703: move failed: path blocked at places.corridor_2`

What a good error message contains

1. The code and line number.
2. What went wrong, in plain words.
3. What the robot can do instead, or a suggested fix.

Runtime errors inside a program

Inside an `on fail:` block, the variable `error` holds the failure:

```
move rover.base to places.lab
on fail:
    log error.code, error.reason
    say "I could not reach the lab: {error.reason}"
    move rover.base to places.dock
```

If no `on fail` handles an error, the robot stops safely and the program ends with the error reported to the operator.

Error messages in the user's language

Message texts are stored in translation tables, so the same `E401` can be shown in English, Arabic or Hindi. Codes never change between languages, which keeps logs searchable.

9. The URITP protocol

The protocol is what passes between a URITP program and a robot. It has three parts: the capability manifest a robot publishes, the instruction set it accepts, and the conformance rules it must follow.

9.1 Capability manifest

Every URITP-compatible robot publishes a manifest describing exactly what it can do. A manifest is a `.uritp` file that begins with `manifest`, written in URITP's own declaration syntax:

```
manifest "ExampleCorp X7 Arm":
    uritp 0.1
    frame tool: pose6d reach 0.85m max_speed 1.5m/s payload 5kg
    actuator gripper: states [open, closed] max_force 140N
    sensor camera: rgb
    sensor force: wrench
    profiles [core, manipulation, vision]
    extension examplecorp.adaptive_weld(seam: Route, current:
Quantity<Current>)
    safety max_speed 1.5m/s
```

Entry	Syntax	Meaning
`manifest`	`manifest "ROBOT NAME": +` block	Starts a manifest; one per robot model.
`uritp`	`uritp VERSION`	The URITP version the robot

Entry	Syntax	Meaning
		implements.
`frame`	`frame NAME: TYPE [reach Q] [max_speed Q] [payload Q]`	A controllable frame. TYPE is `pose2d` (plane), `pose3d` (position in space) or `pose6d` (position and orientation).
`actuator`	`actuator NAME: states [LIST] [max_force Q]`	A part with discrete states.
`sensor`	`sensor NAME: KIND [dimension D]`	A sensor, with the dimension of its readings where it has one.
`profiles`	`profiles [LIST]`	The standard profiles the robot supports (section 7). Must include `core`.
`extension`	`extension VENDOR.NAME(PARAM: TYPE, ...)`	A vendor extension and its typed parameters.
`safety`	`safety max_speed Q` / `safety forbidden_zone ZONE`	Limits enforced on every program.

9.2 Instruction set

Every URITP program is equivalent to a sequence of the instructions below. A compatible robot receives these instructions, never URITP source text directly.

Instruction	Fields	Meaning
`MOVE`	frame, target, speed, tolerance, via, avoid	Goal on a frame
`DRIVE` / `TURN`	frame, distance or angle, speed	Relative base motion
`ACTUATE`	actuator, state, force	Change an actuator state
`SKILL`	name, args	Profile skill: pick, place, pour, weld, spray, takeoff, land, return_home
`SENSE`	sensor	Return the latest reading
`SAY` / `CONFIRM` / `ASK`	text, lang	Talk to a person
`WAIT`	duration or condition, timeout	Pause
`EXT`	name, args	Vendor extension
`NATIVE`	language, code	Escape hatch
`STOP`	none	Stop all motion immediately
`STATUS`	task	Report the state of a task

Instructions are written one per line, as the instruction name followed by `field=value` pairs. Every quantity is expressed in SI units (metres, seconds, radians, kilograms, newtons, amperes, litres), so no robot has to convert units:

```
MOVE frame=tool target=cup.above(0.10) speed=0.2 tolerance=0.005
ACTUATE actuator=gripper state=closed
```

Control flow (`if`, loops, `on fail`, tasks, variables) is resolved before instructions reach the robot. The robot only ever sees the instructions above.

9.3 Conformance rules

A robot may be called URITP-compatible only if it meets every rule below.

1. **Manifest.** It publishes a correct manifest, and its manifest lists every frame, actuator, sensor, profile and extension it accepts.
2. **Core.** It accepts every `core` instruction, plus every instruction of each profile it declares.
3. **Task lifecycle.** For every instruction it reports `accepted`, then `running`, then exactly one of `succeeded`, `failed(reason)` or `cancelled`.
4. **No silent approximation.** It rejects any instruction it cannot perform exactly as written, with an E4xx reason. It never performs a different motion instead.
5. **Stop.** It obeys `STOP` at any moment, from any state, within its stated stopping time.
6. **Safety envelope.** It enforces its own safety limits even if a program or instruction would break them. A program can never switch safety limits off.
7. **Units.** It interprets every quantity in SI units.
8. **Extensions.** Every vendor-specific feature is named `vendor.feature` and listed in the manifest. No vendor feature may reuse a standard URITP name.
9. **Native code.** If it accepts `NATIVE`, it states which languages in its manifest, and treats a `NATIVE` failure like any other failed task.
10. **Conformance tests.** It passes the official URITP conformance test set for its version and declared profiles.

9.4 Natural language

URITP is designed to be the language that spoken requests are translated into. A request made in any human language (English, Arabic, Hindi and so on) is converted into URITP, checked against the robot's manifest and the safety rules, and only then carried out. Replies to the person use the `lang` tag of `say` and `confirm`. A spoken request can never make a robot do anything a written URITP program could not.